Queue Worker Setup Guide

Laravel background jobs (vendor & refund events) — one worker drains the default `database` queue. Replace **USER** and paths with your own.

A. Shared Hosting — cPanel

1. **Find your PHP binary.**
   cPanel → **Software** → **Select PHP Version** (or MultiPHP Manager). Note the version, e.g. PHP 8.2 → binary is usually:

   ```
   /usr/local/bin/ea-php82
   ```

   If unsure, `/usr/local/bin/php` works on most hosts.

2. **Find your project path.**
   cPanel → **File Manager**. Look for the folder containing `artisan`, e.g.

   ```
   /home/USER/public_html
   ```

3. **Open Cron Jobs.**
   cPanel → **Advanced** → **Cron Jobs**.

4. **Set schedule to every minute.**
   Common Settings → *Once Per Minute*, or type `* * * * *`.

5. **Paste this in the Command field** (one line):

   ```
   /usr/local/bin/php /home/USER/public_html/artisan
   queue:work database --stop-when-empty --max-time=55 --
   tries=5 --timeout=120 >> /home/USER/public_html/storage/
   logs/worker.log 2>&1
   ```

6. **Click "Add New Cron Job".** Done.

How it works: every minute the worker starts, processes all waiting jobs, then exits. Cron restarts it the next minute. Workers never overlap.

Verify: File Manager → open `storage/logs/worker.log` after ~2 minutes. You should see job lines or an empty file (empty = no pending jobs, which is fine).

B. VPS / Dedicated — Supervisor

1. **Install Supervisor** (skip if present):

   ```
   sudo apt update && sudo apt install supervisor -y
   ```

2. **Create the config file:**

   ```
   sudo nano /etc/supervisor/conf.d/sixvalley-worker.conf
   ```

3. **Paste this, then save** (Ctrl+O, Enter, Ctrl+X):

   ```
   [program:sixvalley-worker]
   process_name=%(program_name)s_%(process_num)02d
   command=php /home/USER/project/artisan queue:work
   database --sleep=3 --tries=5 --max-time=3600 --
   timeout=120
   autostart=true
   autorestart=true
   stopwaitsecs=130
   user=USER
   numprocs=1
   redirect_stderr=true
   stdout_logfile=/home/USER/project/storage/logs/
   worker.log
   ```

4. **Load and start it:**

   ```
   sudo supervisorctl reread
   sudo supervisorctl update
   sudo supervisorctl start sixvalley-worker:*
   ```

5. **Check it is running:**

   ```
   sudo supervisorctl status sixvalley-worker:*
   ```

   Expect `RUNNING`.

How it works: the worker stays alive permanently. Supervisor restarts it if it crashes or after the server reboots.

Do not add `--stop-when-empty` here — the worker must stay alive.

After Every Deploy — Both Setups

Run this from your project folder (SSH, or cPanel → Terminal):

```
php artisan queue:restart
```

The running worker keeps executing the **old code** until you do. Jobs are not lost — the worker finishes its current job, exits gracefully, and restarts with the new code.

Flag	What it does	cPanel cron	Supervisor
`database`	Queue connection to read from (the `jobs` table)	yes	yes
`--stop-when-empty`	Exit once the queue is drained, so cron can relaunch cleanly	**yes**	**no**
`--max-time`	Recycle the process after N seconds (frees memory)	55	3600
`--tries`	Attempts before a job moves to `failed_jobs`	5	5
`--timeout`	Kill a single job that hangs longer than N seconds	120	120
`--sleep`	Seconds to wait when the queue is empty before re-checking	–	3

Config note: `--timeout=120` must stay below `retry_after` in `config/queue.php` (default 90). Raise `retry_after` to ~150 so a slow job is not retried while it is still running.